VERIQTA

Bash Scripting for Beginners: Your First 10 Scripts

Learn by Doing | Real Examples | Practical Scripts
Build Skills for Linux, DevOps and Automation



10 PRACTICAL SCRIPTS | **BEGINNER FRIENDLY** | **REAL-WORLD EXAMPLES** | **TROUBLESHOOTING TIPS** | **BEST PRACTICES**

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

The 10 Scripts Learners Will Finish With

	Script	Engineering skill
1	System Information Reporter	Linux inspection
2	Disk Space Warning System	Resource monitoring
3	Automatic File Backup	Data protection
4	Application Process Checker	Process monitoring
5	Website Availability Checker	HTTP diagnostics
6	Log Error Investigator	Log analysis
7	Safe Temporary File Cleaner	Controlled maintenance
8	Linux Service Status Reporter	Service troubleshooting
9	Server Connectivity Tester	Network diagnostics
10	Automated Server Health Report	Combined automation

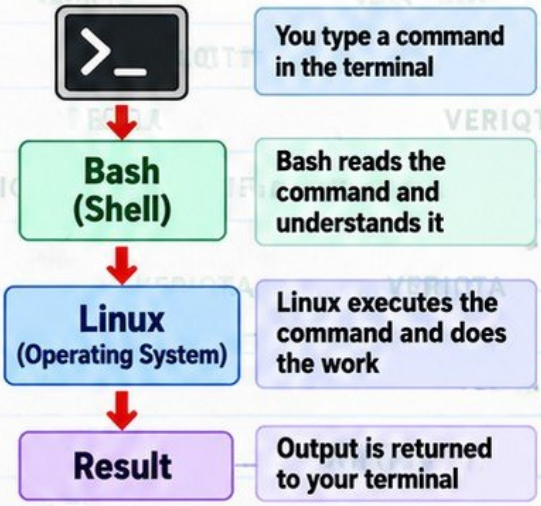
01 What Bash Scripting Actually Does

VERIQTA

1. Key Components and How They Work Together

- **Linux** is the operating system that manages hardware, files, processes and everything on the server.
- The **shell** is a program that lets you interact with Linux using commands.
- **Bash** (Bourne Again Shell) is the most popular shell on Linux. It reads your commands and runs them for you.
- **Terminal** is the interface where you type commands for the shell.
- A **script** is a text file that contains multiple Bash commands. The shell reads the file and runs the commands automatically.

2. How the Pieces Fit Together



3. From Manual Commands to a Reusable Script

Running commands manually (repetitive)

```
user@server:~$ mkdir logs
user@server:~$ cd logs
user@server:~/logs$ touch app.log
user@server:~/logs$ ls -l
```

Turn into a script

Bash script (reusable)

```
#!/usr/bin/env bash
mkdir logs
cd logs
touch app.log
ls -l
```

4. How a Script Works (Simple Flow)



5. DevOps Connection

- Engineers automate repetitive tasks to save time
- Scripts reduce human error.
- Scripts help with server setup, deployments, backups, monitoring and maintenance.
- Automation makes infrastructure management consistent and reliable.
- Bash scripting is widely used in DevOps, cloud engineering, CI/CD pipelines and system administration.

6. Key Takeaway

- A Bash script is a simple text file with Linux commands.
- It helps you do in seconds what would take many manual steps.
- It is a core skill for Linux, DevOps and automation jobs.

02 Your First Bash File: From Text to Execution

VERIQTA

1. Create your first script file

1. Open a terminal.
2. Create a file called **hello.sh**

```
user@server:~$ nano hello.sh
```

This opens a text editor where you write your script.

2. Write a simple script

```
1  #!/usr/bin/env bash
2  echo "Hello, VERIQTA!"
3  echo "This is my first script."
```

- Line 1 is called the **shebang (#!)**. It tells the operating system to use Bash to run this file.
- Lines 2 and 3 are Linux commands.
- Save and close the file (in nano: **CTRL + O**, then **ENTER**, then **CTRL + X**).

3. How the operating system starts a script

You run ./hello.sh → **OS reads the first line (#!)** → **OS uses Bash to run the commands** → **The commands execute and you see the result**

4. Run the script using different methods

1	Run with bash	`user@server:~$ bash hello.sh`
		`Hello, VERIQTA!`
		`This is my first script.`
			• Uses Bash directly.
			• Works even if the file is not executable.

2	Make it executable	`user@server:~$ chmod +x hello.sh`	• Gives execution permission to the owner.

3	Run directly	`user@server:~$ ./hello.sh`
		`Hello, VERIQTA!`
		`This is my first script.`
			• Runs the script using the shebang.
			• This is how you normally run your own scripts.

5. Beginner mistake: Permission denied

```
user@server:~$ ./hello.sh
bash: ./hello.sh: Permission denied
```

- This happens because the file does not have execution permission.
- Linux will not run a file directly (./hello.sh) unless it is marked as executable.
- Fix it with: **chmod +x hello.sh**

6. Understand execution permissions

```
- r w - r - - r - -
```

- Owner (read, write, execute)
- Group (read only)
- Others (read only)

- To run a script, the owner must have execute permission (x).
- **chmod +x** gives execute permission to the owner.
- You can also use: **chmod u+x hello.sh**

7. Key Takeaways

- A Bash script is a text file with Linux commands.
- The shebang (#!/usr/bin/env bash) tells the OS to use Bash.
- You can run a script with **bash hello.sh** or make it executable and use **./hello.sh**.
- Execution permissions control who can run the file.
- Permission denied means the file is not executable.

03 Variables, Quotes, and Command Substitution

VERIQTA

1. What are variables?

- A variable stores a value that you can reuse in your script.
- You can store text, numbers or the output of a command.
- Use a variable when a value might change.

```
1  name="VERIQTA"
2  echo "$name"
3  echo "${name}"
```

→ Set a variable
→ Use the variable
→ Use curly braces (recommended)

2. Spaces around assignment

- Do NOT put spaces around = when assigning a value.
- Correct: name="VERIQTA"
- Incorrect: name = "VERIQTA" (this will cause an error)

✗ Incorrect (spaces)

```
name = "VERIQTA"
bash: name: command not found
```

✓ Correct (no spaces)

```
name="VERIQTA"
```

3. Single vs Double quotes

Type	Example	What it does	When to use
Single quotes ' '	name='VERIQTA"	Treats everything as plain text. Does not expand variables or commands.	Use when you want the exact text.
Double quotes " "	name="VERIQTA"	Allows variable expansion and command substitution.	Use in most cases.

4. Command substitution

- Lets you store the output of a command in a variable.
- Use $(command) or backticks `command`.

```
1  today=$(date)
2  echo "$today"
```

Example output:

```
Wed Oct 15 12:34:56 UTC 2026
```

5. Why quoting variables matters

- If a variable contains spaces, not quoting it can cause problems.
- Always use quotes when using variables.

```
name="VERIQTA Academy"
echo $name    # Wrong
echo "$name"  # Correct
```

Without quotes:
- Bash treats each word as a separate value.
- Can cause unexpected errors.

With quotes:
- Preserves the entire value.
- Keeps spaces and special characters safe.

6. Practice: Get real system information

Write a script or run these commands to display your current user, hostname and date.

```
1  echo "Current user: $(whoami)"
2  echo "Hostname: $(hostname)"
3  echo "Current date: $(date)"
```

Example output:

```
Current user: veriqta
Hostname: server1
Current date: Wed Oct 15 12:34:56 UTC 2026
```

7. Common mistakes

1. Putting spaces around =
2. Forgetting to quote variables
3. Using single quotes when you need expansion
4. Forgetting to use ${name} in complex cases
5. Confusing command substitution with a literal string

8. Key takeaways

- ✓ Variables store values you can reuse.
- ✓ Do not put spaces around = when assigning.
- ✓ Use double quotes for most cases.
- ✓ Use $(command) to capture command output.
- ✓ Always quote variables to avoid unexpected errors.

04 Inputs, Arguments, and Exit Codes

VERIQTA

1. Taking input from the user (interactive input)

- Use `read -r` to get input from the user.
- The script pauses and waits for you to type.
- Useful for simple interactive scripts.

```
#!/usr/bin/env bash
read -r name
echo "Hello, $name!"
```

Example run:

```
$ bash greeting.sh
Enter your name: VERIQTA
Hello, VERIQTA!
```

2. Using command-line arguments

- Arguments are values you pass when running the script.
- `$1`, `$2`, `$3` ... represent the arguments.
- `$#` shows how many arguments were given.

```
#!/usr/bin/env bash
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $#"
```

Example run:

```
$ bash args.sh server1 8080
First argument: server1
Second argument: 8080
Total arguments: 2
```

3. Understanding exit codes ($? and exit)

- Every command and script returns an exit status code.
- 0 means success.
- A non-zero code means something went wrong.
- Use `exit` to set the exit code for your script.

```
#!/usr/bin/env bash
if [ -f "$1" ]; then
  echo "File exists."
  exit 0  # success
else
  echo "File not found."
  exit 1  # error
fi
```

exit 0
Success

- Script completed as expected.
- Used by automation systems to detect success.

exit 1 (or any non-zero code)
Error

- Something went wrong.
- Used by automation systems to detect failure.

4. Interactive input vs Arguments

Interactive input	Command-line arguments
Uses `read -r`	Uses `$1, $2,...`
User types the value when the script runs	Values are provided in the command
Good for simple, one-time input	Better for automation and repeatable tasks
Example: `read -r name`	**Example:** `bash script.sh server1 8080`

5. DevOps connection

- Automation tools (such as CI/CD pipelines, cron, and monitoring systems) run scripts and check the exit code.
- If the exit code is 0, the tool assumes the script succeeded.
- If the exit code is non-zero, the tool assumes the script failed and can take action (such as stopping a deployment or sending an alert).



6. Practice

Create a script called `user-info.sh` that accepts your name as an argument and prints your current user, hostname and date.

```
#!/usr/bin/env bash
echo "Name: $1"
echo "User: $(whoami)"
echo "Hostname: $(hostname)"
echo "Date: $(date)"
```

Example run:

```
$ bash user-info.sh VERIQTA
Name: VERIQTA
User: veriqta
Hostname: server1
Date: Wed Oct 15 12:34:56 UTC 2026
```

7. Key takeaways

- Use `read -r` for interactive input.
- Use `$1`, `$2`, ... for command-line arguments.
- Use `$#` to check how many arguments were provided.
- Use `$?` to check the exit status of the last command.
- Use `exit` to set the exit code of your script.
- Exit code 0 means success, non-zero means failure.
- Automation systems rely on exit codes to decide the next action.

05 Decisions, Loops, and Functions

VERIQTA

1. if statement (make decisions)

- Use **if** to run different commands based on a condition.
- Use **[[ ]]** for safer and more advanced testing.

```bash
#!/usr/bin/env bash
name="VERIQTA"
if [[ -n "$name" ]]; then
    echo "Name is set: $name"
else
    echo "Name is empty."
fi
```

What is happening?

- `[[ -n "$name" ]]` checks that the variable is not empty.
- If true, it prints the name.
- If false, it prints a different message.

Example output:

```
Name is set: VERIQTA
```

2. Comparing values

- Use **-eq**, **-ne**, **-gt**, **-lt** for numbers.
- Use **=** or **!=** for strings.
- Use **[[ ]]** to avoid common errors.

```bash
age=18
if [[ $age -ge 18 ]]; then
    echo "You are an adult."
else
    echo "You are a minor."
fi
```

Example output:

```
You are an adult.
```

3. for loop (repeat a set of commands)

- Use **for** to loop through a list of items.
- Useful for processing files, users, or a list of values.

```bash
#!/usr/bin/env bash
for name in Alice Bob Charlie; do
    echo "Hello, $name!"
done
```

Example output:

```
Hello, Alice!
Hello, Bob!
Hello, Charlie!
```

4. while loop (repeat while a condition is true)

- Use **while** to repeat commands until a condition is no longer true.
- Common in monitoring and waiting tasks.

```bash
#!/usr/bin/env bash
count=1
while [[ $count -le 3 ]]; do
    echo "Count: $count"
    ((count++))
done
```

Example output:

```
Count: 1
Count: 2
Count: 3
```

5. Functions (organize reusable logic)

- Use functions to group commands you can reuse.
- Helps keep your script clean and easy to maintain.

```bash
#!/usr/bin/env bash
greet() {
    local name="$1"
    echo "Hello, $name!"
}

greet "VERIQTA"
greet "Engineer"
```

Example output:

```
Hello, VERIQTA!
Hello, Engineer!
```

Why use functions?

- Reuse code
- Make scripts shorter
- Easier to update
- Useful in larger scripts

6. Common mistakes to avoid

Using -eq with strings (wrong)

```bash
if [[ "a" -eq "b" ]]; then   # Error
```

Using = with numbers (not recommended)

```bash
if [ 5 = 5 ]; then   # Works, but not ideal
```

Forgetting to close if, for or while

```bash
if [[ -n "$name" ]]; then
    echo "Hello"  # Missing fi
```

Using unquoted variables with spaces

```bash
file="My File.txt"
echo $file     # Will break
echo "$file"   # Correct
```

7. Connected example: Use everything together

- This script uses a function, takes a user input, checks a condition, and loops.
- Save as **demo.sh** and run it: **bash demo.sh**

```bash
#!/usr/bin/env bash
greet() { echo "Hello, $1!"; }
read -r name
if [[ -z "$name" ]]; then echo "No name provided. Exiting,"; exit 1; fi
greet "$name"
for i in 1 2 3; do echo "Processing item $i for $name"; done
```

Example run:

```
$ bash demo.sh
Enter your name: VERIQTA
Hello, VERIQTA!
Processing item 1 for VERIQTA
Processing item 2 for VERIQTA
Processing item 3 for VERIQTA
```

06 Script 1: System Information Reporter

VERIQTA

Filename: system-info.sh

1. What this script does

- Displays important system information.
- Shows the hostname, current user, kernel version, uptime and available memory.
- Helps you quickly understand the server during troubleshooting.

2. Commands used and what they do

Command	What it shows
hostname	System hostname
whoami	Current logged in user
uname -r	Kernel version
uptime	How long the system has been running
free -h	Memory usage in human readable format

3. The complete script: system-info.sh

```
#!/usr/bin/env bash

echo "========== System Information =========="
echo "Hostname       : $(hostname)"
echo "Current User   : $(whoami)"
echo "Kernel Version: $(uname -r)"
echo "Uptime         : $(uptime)"

echo ""
echo "Memory Usage:"
free -h
```

4. How to run the script

1. Create the file — `nano system-info.sh`
2. Make it executable — `chmod +x system-info.sh`
3. Run the script — `./system-info.sh`

Alternative (no execute permission required)

`bash system-info.sh`

5. Example output

```
========== System Information ==========
Hostname       : server1
Current User   : veriqta
Kernel Version: 6.8.0-45-generic
Uptime         : 2 days, 4:15, 1 user,  load average: 0.00, 0.01, 0.05

Memory Usage:
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       1.2Gi       4.8Gi       120Mi       1.6Gi       6.1Gi
Swap:          2.0Gi          0B       2.0Gi
```

6. Each command separately

hostname	server1
whoami	veriqta
uname -r	6.8.0-45-generic
uptime	2 days, 4:15, 1 user, load average: 0.00, 0.01, 0.05
free -h	total used free Mem: 7.6Gi 1.2Gi 4.8Gi Swap: 2.0Gi 0B 2.0Gi

7. Real-world use

- Collect basic system information during server troubleshooting.
- Useful when investigating performance issues.
- Helps you quickly understand the server environment.
- Can be used in monitoring scripts or incident response.



8. Key takeaways

- **hostname** shows the system hostname.
- **whoami** shows the current user.
- **uname -r** shows the kernel version.
- **uptime** shows how long the system has been running.
- **free -h** shows memory usage.
- This script gives a quick overview of the server.

08 Script 3: Automatic File Backup

VERIQTA

Filename: backup-files.sh

1. What this script does

- Creates a timestamped archive of a specified directory.
- Saves the backup in a dedicated backup folder.
- Helps protect important files before making changes.
- Useful for configuration files, application data, or any important directory.

2. Commands used and what they do

Command	What it does
`mkdir -p`	Creates a directory (including parent directories if they do not exist)
`tar -czf`	Creates a compressed archive (.tar.gz)
`date`	Generates a timestamp for the filename
`basename`	Gets the last part of a directory path
`dirname`	Gets the parent directory path

3. The complete script: backup-files.sh

```
#!/usr/bin/env bash

# Check if a directory was provided
if [ -z "$1" ]; then
  echo "Usage: $0 <directory-to-backup>"
  exit 1
fi

SOURCE_DIR="$1"
DIR_NAME=$(basename "$SOURCE_DIR")
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="$HOME/backups"
mkdir -p "$BACKUP_DIR"
BACKUP_FILE="$BACKUP_DIR/${DIR_NAME}_$TIMESTAMP.tar.gz"

echo "Creating backup..."
tar -czf "$BACKUP_FILE" -C "$(dirname "$SOURCE_DIR")"
    "$DIR_NAME"

if [ $? -eq 0 ]; then
  echo "Backup created successfully:"
  echo "$BACKUP_FILE"
else
  echo "Backup failed."
  exit 1
fi
```

4. How to run the script

1. Create the file — `nano backup-files.sh`
2. Make it executable — `chmod +x backup-files.sh`
3. Run the script (provide a directory) — `./backup-files.sh /etc`

Example:

```
./backup-files.sh /etc/nginx
```

5. Example output

```
$ ./backup-files.sh /etc/nginx
Creating backup...
Backup created successfully:
/home/veriqta/backups/nginx_20261015_123456.tar.gz
```

6. Verify the backup

- List the backup file:

```
ls -lh ~/backups
```

- Check the contents:

```
tar -tzf ~/backups/nginx_20261015_123456.tar.gz
```

7. Real-world use

- Back up configuration files before making changes.
- Create regular backups of important directories.
- Useful in servers, cloud instances and DevOps environments.
- Can be scheduled with cron for automatic backups.



8. Security and best practices

- Always verify the backup file after creation.
- Use descriptive filenames with timestamps.
- Store backups in a safe location.
- Do not back up sensitive files without proper access control.
- Test restoring the backup to confirm it works.
- Monitor available disk space.
- Consider encrypting backups for sensitive data.

09 Script 4: Application Process Checker

VERIQTA

Filename: process-check.sh

1. What this script does

- Checks whether a specified application process is running.
- Finds the process ID (PID) if it is running.
- Uses conditional exit codes to indicate success or failure.
- Useful for basic operational checks on a Linux server.

2. Commands used and what they do

Command	What it does
pgrep	Searches for a process by name and returns the PID(s).
ps -ef	Shows detailed information about running processes.
echo	Prints messages to the terminal.
if	Checks a condition.
exit	Sets the exit code for the script.

3. The complete script: process-check.sh

```
#!/usr/bin/env bash

# Check if a process name was provided
if [ -z "$1" ]; then
    echo "Usage: $0 <process-name>"
    exit 1
fi

PROCESS_NAME="$1"

# Check if the process is running
PIDS=$(pgrep -f "$PROCESS_NAME")

if [ -n "$PIDS" ]; then
    echo "Process '$PROCESS_NAME' is running."
    echo "PID(s): $PIDS"
    exit 0   # success
else
    echo "Process '$PROCESS_NAME' is not running."
    exit 1   # not found
fi
```

4. How to run the script

1. Create the file — `nano process-check.sh`
2. Make it executable — `chmod +x process-check.sh`
3. Run the script (provide a process name) — `./process-check.sh nginx`

5. Example output

When the process is running:

```
$ ./process-check.sh nginx
Process 'nginx' is running.
PID(s): 1234 1256
```

Exit code: 0 (Success)
`echo $?`
0

When the process is not running:

```
$ ./process-check.sh apache
Process 'apache' is not running.
```

Exit code: 1 (Not found)
`echo $?`
1

6. Understanding the process ID (PID)

```
$ pgrep -f nginx
1234
1256
```

- pgrep returns the PID(s) of the matching process.
- A PID is a unique number that identifies a running process.
- There can be multiple PIDs if the application runs multiple processes.

7. Real-world use

- Quickly check if important applications (nginx, apache, mysql, redis, etc.) are running.
- Useful for monitoring scripts, cron jobs, or simple health checks.
- Helps detect if a service has stopped unexpectedly.
- Can be used as part of a larger automation or alerting system.



8. Production awareness

- A running process does not necessarily mean the application is healthy.
- The process could be stuck, misbehaving, or not serving requests.
- For a real health check, also verify logs, service response, or application-specific endpoints.
- Use this script as a basic check, then add deeper checks for production environments.

9. Key takeaways

- Use pgrep to find running processes.
- A PID is a unique number for a process.
- Use conditional logic to handle success and failure.
- Use exit codes so automation systems can detect the result.
- Useful for basic operational checks on a Linux server.
- A running process does not guarantee application health.

10 Script 5: Website Availability Checker

VERIQTA

Filename: website-check.sh

1. What this script does

- Checks whether a website responds to an HTTP request.
- Displays the HTTP status code and response time.
- Uses a timeout to avoid waiting too long.
- Uses exit codes to indicate success or failure.
- Useful for basic website or API availability checks.

2. Commands used and what they do

Command	What it does
curl	Sends an HTTP request to a website.
-s	Silent mode (no progress output).
-o /dev/null	Discards the response body.
-w	Outputs specific data (status code, time).
--max-time	Sets a request timeout in seconds.
echo	Prints messages to the terminal.
if	Checks a condition.
exit	Sets the exit code for the script.

3. The complete script: website-check.sh

```
#!/usr/bin/env bash

# Check if a URL was provided
if [ -z "$1" ]; then
    echo "Usage: $0 <website-url>"
    exit 1
fi
URL="$1"
TIMEOUT=10

echo "Checking $URL ..."

# Get HTTP status code and response time
RESPONSE=$(curl -s -o /dev/null -w "%{http_code} %{time_total}" \
      --max-time $TIMEOUT "$URL")
HTTP_CODE=$(echo "$RESPONSE" | awk '{print $1}')
RESPONSE_TIME=$(echo "$RESPONSE" | awk '{print $2}')

if [ "$HTTP_CODE" -ge 200 ] && [ "$HTTP_CODE" -lt 400 ]; then
    echo "Website is reachable!"
    echo "HTTP Status Code: $HTTP_CODE"
    echo "Response Time: ${RESPONSE_TIME}s"
    exit 0   # success
else
    echo "Website is not reachable or returned an error."
    echo "HTTP Status Code: $HTTP_CODE"
    echo "Response Time: ${RESPONSE_TIME}s"
    exit 1   # failure
fi
```

4. How to run the script

1. Create the file — `nano website-check.sh`
2. Make it executable — `chmod +x website-check.sh`
3. Run the script (provide a URL) — `./website-check.sh https://veriqta.com`

5. Example output

When the website is reachable:

```
$ ./website-check.sh https://veriqta.com
Checking https://veriqta.com ...
Website is reachable!
HTTP Status Code: 200
Response Time: 0.42s
```

Exit code: 0 (Success)
HTTP 200–399 means the server responded.

When the website is not reachable:

```
$ ./website-check.sh https://example.org
Checking https://example.org ...
Website is not reachable or
returned an error.
HTTP Status Code: 000
Response Time: 10.00s
```

Exit code: 1 (Failure)
000 usually means the server did not responed (e.g. timeout).

6. Common HTTP status codes

Status code	Meaning
200	OK (success)
301	Moved permanently (redirect)
400	Bad request (client error)
401	Unauthorized
403	Forbidden
404	Not found
500	Internal server error
503	Service unavailable

7. Real-world use

- Check if websites or APIs are available.
- Useful for monitoring scripts, cron jobs, or incident checks.
- Helps detect downtime quickly.
- Can be extended to check multiple URLs.



8. Common mistakes to avoid

- Assuming every HTTP response means the application is healthy.
- Not using a timeout (the script may hang).
- Not checking the HTTP status code.
- Assuming only 200 means success (3xx redirects are also successful).
- Not handling network errors (e.g. 000).

9. Key takeaways

- Use curl to send HTTP requests.
- Check HTTP status codes to determine the result.
- Use a timeout to avoid long waits.
- Use exit codes for automation.
- A response does not always mean the application is healthy.

11 Script 6: Log Error Investigator

VERIQTA

Filename: log-investigator.sh

1. What this script does

- Reads a log file and counts lines containing error indicators.
- Helps you quickly identify application errors during an incident.
- Uses grep and `wc -l` to search and count matches.
- Shows sample error lines for investigation.

2. Commands used and what they do

Command	What it does
grep	Searches for a pattern in a file.
grep -i	Case-insensitive search.
wc -l	Counts the number of lines.
head	Shows the first few lines of output.
tail	Shows the last few lines of a file.
cat	Displays the content of a file.

3. The complete script: log-investigator.sh

```
#!/usr/bin/env bash

# Check if a log file was provided
if [ -z "$1" ]; then
    echo "Usage: $0 <log-file> [keyword]"
    exit 1
fi

LOG_FILE="$1"
KEYWORD="${2:-error}"   # default keyword is 'error'

# Check if the file exists and is readable
if [ ! -r "$LOG_FILE" ]; then
    echo "Error: Cannot read file $LOG_FILE"
    exit 1
fi

echo "Log File : $LOG_FILE"
echo "Keyword  : $KEYWORD"
echo "========================="

# Count matching lines (case-insensitive)
COUNT=$(grep -i "$KEYWORD" "$LOG_FILE" | wc -l)

echo "Total matching lines: $COUNT"
echo ""
echo "First 5 matching lines:"
grep -i "$KEYWORD" "$LOG_FILE" | head -n 5
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOG_FILE" | tail -n 5

if [ "$COUNT" -gt 0 ]; then
    exit 1  # errors found
else
    exit 0  # no errors found
fi
```

4. How to run the script

1. Create the file — `nano log-investigator.sh`
2. Make it executable — `chmod +x log-investigator.sh`
3. Run the script (with a log file) — `./log-investigator.sh /var/log/nginx/error.log`
4. Run with a custom keyword (optional) — `./log-investigator.sh app.log exception`

5. Example log file (sample)

```
2026-10-15 10:20:01 [INFO]  Application started
2026-10-15 10:21:05 [ERROR] Failed to connect to database
2026-10-15 10:22:10 [WARN]  Slow response time
2026-10-15 10:23:11 [ERROR] Connection timeout
2026-10-15 10:24:00 [INFO]  User login successful
2026-10-15 10:25:30 [ERROR] Failed to write to disk
```

6. Example output

```
$ ./log-investigator.sh app.log error
Log File   : app.log
Keyword    : error
=========================
Total matching lines: 3

First 5 matching lines:
2026-10-15 10:21:05 [ERROR] Failed to connect to database
2026-10-15 10:23:11 [ERROR] Connection timeout
2026-10-15 10:25:30 [ERROR] Failed to write to disk
Last 5 matching lines:
2026-10-15 10:21:05 [ERROR] Failed to connect to database
2026-10-15 10:23:11 [ERROR] Connection timeout
2026-10-15 10:25:30 [ERROR] Failed to write to disk
```

7. Real-world use

- Quickly identify recurring application errors.
- Useful during incident investigation.
- Helps you understand the frequency of errors.
- Can be extended to send alerts when errors exceed a threshold.



8. Common mistakes

- Using the wrong keyword (may miss errors).
- Search is case-sensitive (without -i).
- Counting harmless messages that contain the keyword.
- Using a log file path that does not exist.
- Assuming no errors means the application is healthy.

9. Troubleshooting notes

- Some errors may use different words (e.g. exception, fail, critical).
- Application logs can be large; use grep and filters wisely.
- Keyword matching may include non-error messages.
- A log showing no errors does not guarantee the application is working correctly.
- Always check timestamps and context around the error lines.

10. Key takeaways

- Use grep to search log files.
- Use `wc -l` to count matching lines.
- Use -i for case-insensitive search.
- Always verify the results manually.
- Keyword search is a starting point, not a complete health check.

12 Script 7: Safe Temporary File Cleaner

VERIQTA

Filename: temp-cleaner.sh

1. What this script does

- Finds old files inside a dedicated practice directory.
- Shows which files would be removed (dry run).
- Uses safe deletion controls and validation checks.
- Helps you understand controlled cleanup and disk housekeeping.
- Prevents accidental deletion of important files.

2. Commands used and what they do

Command	What it does
find	Searches for files and directories.
-type f	Matches only files (not directories).
-mtime	Finds files older than a specified number of days.
-print	Displays the files that match.
-delete	Deletes the files (use with caution).
echo	Prints messages to the terminal.
exit	Sets the exit code for the script.

3. The complete script: temp-cleaner.sh

```
#!/usr/bin/env bash

# Safe temporary file cleaner
if [ -z "$1" ]; then
    echo "Usage: $0 <directory> [days] [--delete]"
    echo "Example: $0 /tmp/practice 7"
    echo "Example (delete): $0 /tmp/practice 7 --delete"
    exit 1
fi

DIR="$1"
DAYS="${2:-7}"    # default 7 days
DELETE_FLAG="$3"

# Validate directory
if [ ! -d "$DIR" ]; then
    echo "Error: Directory '$DIR' does not exist."
    exit 1
fi

echo "Searching for files older than $DAYS days in: $DIR"

if [ "$DELETE_FLAG" = "--delete" ]; then
    echo "Deleting files..."
    find "$DIR" -type f -mtime +"$DAYS" -print -delete
    echo "Cleanup completed."
else
    echo "DRY RUN: The following files would be removed:"
    find "$DIR" -type f -mtime +"$DAYS" -print
    echo "Use --delete to actually remove these files."
fi
```

4. How to run the script

1. Create the file — `nano temp-cleaner.sh`
2. Make it executable — `chmod +x temp-cleaner.sh`
3. Dry run (see what would be deleted) — `./temp-cleaner.sh /tmp/practice 7`
4. Delete the files (after verifying) — `./temp-cleaner.sh /tmp/practice 7 --delete`

5. Example output (dry run)

```
$ ./temp-cleaner.sh /tmp/practice 7
Searching for files older than 7 days in: /tmp/practice
DRY RUN: The following files would be removed:
/tmp/practice/old-log.txt
/tmp/practice/report-2024-09-01.log
/tmp/practice/debug.log
/tmp/practice/temp-file.tmp
Use --delete to actually remove these files.
```

6. Example output (delete)

```
$ ./temp-cleaner.sh /tmp/practice 7 --delete
Searching for files older than 7 days in: /tmp/practice
Deleting files...
/tmp/practice/old-log.txt
/tmp/practice/report-2024-09-01.log
/tmp/practice/debug.log
/tmp/practice/temp-file.tmp
Cleanup completed.
```

7. Real-world use

- Clean up temporary files, old logs, or unused files.
- Helps free up disk space.
- Useful on servers and development environments.
- Can be scheduled with cron for automatic cleanup.



8. Critical safety lesson

- Always do a dry run first. Verify the target directory path.
- Never run deletion commands on production directories.
- Use absolute paths (e.g. /var/log/app) to avoid mistakes.
- Double-check the number of days.
- Be careful with permissions.
- Deleted files may be difficult or impossible to recover.

9. Common mistakes

- Using the wrong directory (e.g. / instead of /tmp/practice).
- Forgetting to do a dry run.
- Using a negative or incorrect -mtime value.
- Not checking the list before deleting.
- Running the script with high permissions (e.g. sudo) without care.
- Assuming deleted files can always be recovered.

10. Key takeaways

- Use find to search for files.
- -type f matches only files.
- -mtime finds files older than a number of days.
- Do a dry run and review the results.
- Validate the directory path.
- Use --delete only after confirming it is safe.
- Never blindly delete production files.

13 Script 8: Linux Service Status Reporter

VERIQTA

Filename: service-status.sh

1. What this script does

- Checks the status of a specified systemd service.
- Shows whether the service is running, stopped, or not found.
- Uses systemctl is-active to check service status.
- Helps you investigate application services that stopped running.
- Useful for basic operational checks on a Linux server.

2. Commands used and what they do

Command	What it does
systemctl	Manages systemd services.
systemctl is-active	Checks if a service is active (running).
systemctl status	Shows detailed information about a service.
echo	Prints messages to the terminal.
if	Checks a condition.
exit	Sets the exit code for the script.

3. The complete script: service-status.sh

```
#!/usr/bin/env bash

# Check if a service name was provided
if [ -z "$1" ]; then
    echo "Usage: $0 <service-name>"
    echo "Example: $0 nginx"
    exit 1
fi

SERVICE="$1"

# Check if the service exists
if ! systemctl list-unit-files | grep -q "^$SERVICE\.service"; then
    echo "Error: Service '$SERVICE' not found."
    exit 1
fi
# Check service status
if systemctl is-active --quiet "$SERVICE"; then
    echo "Service: $SERVICE"
    echo "Status : running (active)"
    exit 0
else
    echo "Service: $SERVICE"
    echo "Status : not running (inactive or failed)"
    echo "Tip    : Use 'systemctl status $SERVICE' for details."
    exit 1
fi
```

4. How to run the script

1. Create the file — `nano service-status.sh`
2. Make it executable — `chmod +x service-status.sh`
3. Run the script (with a service name) — `./service-status.sh nginx`
4. Check detailed status (if needed) — `systemctl status nginx`

5. Example output (service running)

```
$ ./service-status.sh nginx
Service: nginx
Status : running (active)
```

6. Example output (service not running)

```
$ ./service-status.sh nginx
Service: nginx
Status : not running (inactive or failed)
Tip    : Use 'systemctl status nginx' for details.
```

7. Real-world use

- Investigate application services that stopped running.
- Check critical services like nginx, apache, docker, or your own application.
- Useful during incident troubleshooting.
- Can be used in monitoring scripts or health checks.



8. Production awareness

- ⚠ Report the failure first. Do not automatically restart services without understanding the cause.
- ⚠ A service can be running but not working correctly.
- ⚠ Use "systemctl status <service>" to investigate errors.
- ⚠ Some services may fail due to configuration issues, dependency problems, or resource limits.
- ⚠ Always check logs (e.g. journalctl -u <service>) for the real cause.

9. Common mistakes

- ✗ Using the wrong service name.
- ✗ Assuming a running service means the application is healthy.
- ✗ Not checking detailed status or logs.
- ✗ Automatically restarting a failed service.
- ✗ Ignoring permission errors (use sudo if required).

10. Key takeaways

- ✓ Use systemctl is-active to check service status.
- ✓ A service can be active, inactive, failed, or not found.
- ✓ Always investigate the cause before restarting.
- ✓ Use systemctl status and logs for more details.
- ✓ Include service checks in your basic operational scripts.

14 Script 9: Server Connectivity Tester

VERIQTA

Filename: port-check.sh

1. What this script does

- Tests whether a specified TCP port is reachable on an authorized server.
- Uses `nc -z -w 3` to check the connection.
- Shows whether the port is open, closed, or unreachable.
- Helps you diagnose basic network connectivity issues.
- Useful for checking application endpoints (e.g. web server, database, API).

2. Commands used and what they do

Command	What it does
`nc`	Netcat utility for network connectivity testing.
`nc -z`	Checks if a port is open (no data sent).
`-w 3`	Sets a timeout of 3 seconds.
`echo`	Prints messages to the terminal.
`if`	Checks a condition.
`exit`	Sets the exit code for the script.

3. The complete script: port-check.sh

```
#!/usr/bin/env bash

# Check if a hostname and port were provided
if [ "$#" -ne 2 ]; then
   echo "Usage: $0 <hostname|ip> <port>"
   echo "Example: $0 veriqta.com 80"
   exit 1
fi

HOST="$1"
PORT="$2"

# Validate port is a number
if ! [[ "$PORT" =~ ^[0-9]+$ ]]; then
   echo "Error: Port must be a valid number."
   exit 1
fi

# Test connectivity using nc
echo "Checking connectivity to $HOST on port $PORT ..."

if nc -z -w 3 "$HOST" "$PORT" 2>/dev/null; then
   echo "SUCCESS: $HOST:$PORT is reachable (port open)."
   exit 0
else
   echo "FAILED: $HOST:$PORT is not reachable (port closed
   or filtered)."
   exit 1
fi
```

4. How to run the script

1. Create the file — `nano port-check.sh`
2. Make it executable — `chmod +x port-check.sh`
3. Run the script (with hostname and port) — `./port-check.sh example.com 80`
4. Run with an IP address (optional) — `./port-check.sh 192.168.1.10 22`

5. Example output (port open)

```
$ ./port-check.sh veriqta.com 80
Checking connectivity to veriqta.com on port 80 ...
SUCCESS: veriqta.com:80 is reachable (port open).
Exit code: 0
```

6. Example output (port closed or unreachable)

```
$ ./port-check.sh veriqta.com 3306
Checking connectivity to veriqta.com on port 3306 ...
FAILED: veriqta.com:3306 is not reachable
(port closed or filtered).
Exit code: 1
```

7. Real-world use

- Check if web servers (port 80/443) are reachable.
- Verify database ports (e.g. 3306 for MySQL, 5432 for PostgreSQL).
- Useful during incident troubleshooting.
- Helps confirm network connectivity between servers, containers, or cloud instances.

8. Common port numbers

Service	Port
HTTP	80
HTTPS	443
SSH	22
MySQL	3306
PostgreSQL	5432
Redis	6379
Nginx (alt)	8080
Application (example)	3000

9. Beginner mistakes

- Using the wrong hostname or IP address.
- Using the wrong port number.
- Confusing an open port with a fully healthy application.
- Testing a port on a server you are not authorized to access.
- Not considering firewalls or security groups.
- Assuming a slow response always means the port is closed.

10. Key takeaways

- Use `nc -z -w 3` to test TCP port connectivity.
- Provide the correct hostname and port number.
- An open port only means the port is reachable, not that the application is healthy.
- Use this script for basic network troubleshooting.
- Always test on authorized systems only.

15 Script 10: Automated Server Health Report

VERIQTA

Filename: health-report.sh

1. What this script does

- Collects system information, disk usage, memory availability, and process checks.
- Combines the information into a single timestamped report file.
- Uses functions and output redirection.
- Includes basic failure handling.
- Useful for a first-pass diagnostic report during server investigations.

2. Commands used and what they do

Command	What it does
hostname	Shows the server hostname.
whoami	Shows the current user.
uname -r	Shows the kernel version.
df -h	Shows disk usage.
free -h	Shows memory availability.
ps aux	Lists running processes.
date	Outputs the current date and time.
echo	Prints messages to the terminal or file.
> , >>	Redirects output to a file (overwrite/append).
if	Checks a condition.
exit	Sets the exit code for the script.

3. The complete script: health-report.sh

```
#!/usr/bin/env bash

# Automated Server Health Report

REPORT_DIR="$HOME/server-reports"
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_FILE="$REPORT_DIR/health-report-$TIMESTAMP.txt"

# Create report directory
mkdir -p "$REPORT_DIR"

# Start report
{
  echo "=================================="
  echo " Linux Server Health Report"
  echo " Generated: $(date)"
  echo "=================================="
  echo ""

  # System information
  echo "[1] System Information"
  echo "Hostname    : $(hostname)"
  echo "Current User : $(whoami)"
  echo "Kernel Version: $(uname -r)"

  # Disk usage
  echo "[2] Disk Usage"
  df -h

  # Memory information
  echo "[3] Memory Availability"
  free -h

  # Top processes (top 10 by memory usage)
  echo "[4] Top 10 Processes (by memory usage)"
  ps aux --sort=-%mem | head -n 11

  # Check important services (example: nginx and ssh)
  echo "[5] Service Checks"
  for service in nginx ssh; do
    if systemctl is-active --quiet "$service"; then
      echo "$service : running"
    else
      echo "$service : not running (inactive or failed)"
    fi
  done
  echo "=================================="
  echo "Report saved to: $REPORT_FILE"
} > "$REPORT_FILE"
echo "Health report completed: $REPORT_FILE"
```

4. How to run the script

1	Create the file	`nano health-report.sh`
2	Make it executable	`chmod +x health-report.sh`
3	Run the script	`./health-report.sh`

5. Example output (terminal)

```
$ ./health-report.sh
Health report completed: /home/user/server-reports/
health-report-2026-10-15_14-30-22.txt
```

6. Example report file (excerpt)

```
==================================
Linux Server Health Report
Generated: 2026-10-15 14:30:22
==================================
[1] System Information
Hostname    : veriqta-server
Current User : ubuntu
Kernel Version: 6.5.0-101-generic
[2] Disk Usage
Filesystem     Size  Used Avail Use% Mounted on
/dev/sda1      100G  45G   50G  48% /
[3] Memory Availability
               total        used        free      shared  buff/cache   available
Mem:           7.7G        1.2G        4.1G        0.3G        2.4G        5.8G
[4] Top 10 Processes (by memory usage)
USER         PID  %MEM  %CPU  COMMAND
ubuntu      1234   5.0   1.2  python3
root         567   3.1   0.8  java
...
[5] Service Checks
nginx : running
ssh   : running

==================================
```

7. Real-world use

- Get a quick overview of server health.
- Useful during incident investigations.
- Share a single report file with your team.
- Can be scheduled with cron for health checks.

8. Important notes

- The report is a first-pass diagnostic. Investigate any issues found.
- Customize the service list for your environment.
- You can extend this script to include more checks (e.g. disk space thresholds).

9. Beginner mistakes

- Assuming a server is healthy just because services are running.
- Not checking the actual report file.
- Ignoring warnings or errors in the output.
- Using this script without understanding what the results mean.

10. Key takeaways

- Learned to combine multiple checks in one script.
- Used functions and output redirection.
- Created timestamped report files.
- Added basic failure handling.
- Helps with real-world server troubleshooting.

16 Debugging Bash Scripts Like an Engineer

VERIQTA

Filename: (use any script, e.g. debug-example.sh)

1. What this topic covers

- Learn `bash -n`, `bash -x`, `printf`, and checking exit codes.
- Investigate syntax errors, empty variables, missing files, and failed commands.
- Practice by intentionally breaking a copy of one earlier script, finding the cause, correcting it, and verifying the fix.
- Develop a systematic approach to debugging bash scripts like an engineer.

2. Commands used and what they do

Command	What it does
`bash -n <file>`	Checks script syntax without running it.
`bash -x <file>`	Runs the script with debug output (showing each command).
`printf`	Prints formatted output (more reliable than echo).
`$?`	Shows the exit code of the last command.
`set -e`	Exit immediately if a command fails.
`set -u`	Treat unset variables as an error.
`test -f <file>`	Checks if a file exists.
`test -z <var>`	Checks if a variable is empty.
`||`	Run a command if the previous one fails.

3. Example script: debug-example.sh

```
#!/usr/bin/env bash

# Simple script to demonstrate debugging
FILE="$1"
if [ -z "$FILE" ]; then
   printf "Usage: %s <filename>\n" "$0"
   exit 1
fi

# Check if file exists
if [ ! -f "$FILE" ]; then
   printf "Error: File '%s' does not exist.\n" "$FILE"
   exit 1
fi

# Show file details
printf "File: %s\n" "$FILE"
printf "Size: %s bytes\n" "$(stat -c%s "$FILE")"
printf "Last modified: %s\n" "$(stat -c%y "$FILE")"
exit 0
```

4. How to run and debug the script

1	Check syntax (without running)	`bash -n debug-example.sh`
2	Run with debug output	`bash -x debug-example.sh test.txt`
3	Run normally	`./debug-example.sh test.txt`
4	Check exit code (after running)	`echo $?`
5	Use printf for clear output	`printf "Message: %s\n" "$VAR"`

5. Common errors and how to fix them

Problem	Example error/output	Fix
Syntax error	bash: line 5: syntax error	Run bash -n, check missing quotes, brackets or fi.
Empty variable	File '' does not exist	Check with [-z "$VAR"] and add validation.
Missing file	No such file or directory	Use [-f "$FILE"] to check before using the file.
Failed command	Command returned error	Check exit code $? and handle the failure.

6. Practice: break, debug and fix

1	Make a copy	`cp debug-example.sh broken.sh`
2	Introduce a bug (e.g. missing fi)	`# remove the last fi from broken.sh`
3	Check syntax	`bash -n broken.sh`
4	Fix the error	Add the missing `fi` back.
5	Verify the fix	`bash -n broken.sh && ./broken.sh test.txt`

7. Real-world use

- Debug scripts during automation and troubleshooting.
- Identify why a script failed in production logs.
- Save time by finding the real cause quickly.
- Build more reliable and maintainable scripts.



8. Troubleshooting tips

- ⚠ Always check syntax with `bash -n` before running.
- ⚠ Use `bash -x` to see exactly which command fails.
- ⚠ Validate all input variables.
- ⚠ Check file paths and permissions.
- ⚠ Look at the exit code `$?` after every important command.
- ⚠ Use meaningful error messages with `printf`.

9. Beginner mistakes

- ✗ Forgetting to close `if`, `fi` or `do`.
- ✗ Not quoting variables.
- ✗ Using undefined variables (empty values).
- ✗ Assuming a command succeeded without checking `$?`.
- ✗ Ignoring error messages.
- ✗ Testing with the wrong file path.

10. Key takeaways

- ✓ Use `bash -n` to check syntax.
- ✓ Use `bash -x` to trace execution.
- ✓ Use `printf` for clear and consistent output.
- ✓ Always check exit codes.
- ✓ Validate inputs and file paths.
- ✓ Practice breaking and fixing scripts to build debugging skills.

17 Making Scripts Safer and More Reliable

VERIQTA

Filename: safe-script.sh

1. What this topic covers

- Learn input validation, quoted variables, explicit error handling, and safe file permissions.
- Understand **set -euo pipefail** and its limitations.
- Make scripts safer, more reliable, and easier to maintain.
- Avoid common security risks in bash scripts.
- Build habits used in real-world production environments.

2. Key commands and features

Command/Feature	What it does
`set -e`	Exit immediately if a command fails.
`set -u`	Treat unset variables as an error.
`set -o pipefail`	Return the exit code of the first failed command in a pipeline.
`read -r`	Read user input safely (no backslash interpretation).
`"$var"`	Use quoted variables to prevent word splitting and globbing.
`chmod`	Set safe file permissions on scripts.
`if [ -z "$var" ]`	Check for empty input.
`||`	Handle errors and run alternative commands.

3. Example script: safe-script.sh

```
#!/usr/bin/env bash

# Safer script with input validation and error handling
set -euo pipefail

# Check if a directory was provided
if [ $# -ne 1 ]; then
    echo "Usage: $0 <directory>"
    exit 1
fi

DIR="$1"

# Validate input
if [ ! -d "$DIR" ]; then
    echo "Error: '$DIR' is not a valid directory."
    exit 1
fi

# Example operation: list files safely
echo "Listing files in: $DIR"
if ! ls -la "$DIR" > /tmp/safe-script-output.txt 2>&1; then
    echo "Error: Failed to list files in '$DIR'."
    exit 1
fi

echo "Success: Output saved to /tmp/safe-script-output.txt"
```

4. How to run the script

1	Create the file	`nano safe-script.sh`
2	Make it executable	`chmod +x safe-script.sh`
3	Run with a directory	`./safe-script.sh /var/log`
4	Try with a missing directory (to see error)	`./safe-script.sh /notfound`
5	Check the output file	`cat /tmp/safe-script-output.txt`

5. Using set -euo pipefail

```
# Exit on error, unset variable or pipeline failure
set -euo pipefail
```

What each option does:

- `-e` : Exit immediately if any command fails.
- `-u` : Treat unset variables as an error.
- `-o pipefail` : In a pipeline, return the exit code of the first failed command.

Important:

- ✗ It does not replace deliberate error handling.
- ✗ It does not catch all errors (e.g. errors in some conditionals).
- ✗ You still need to validate inputs and handle errors properly.

6. Best practices

- ✓ Always validate user input.
- ✓ Use quoted variables: `"$var"`.
- ✓ Handle errors with `if`, `||`, or custom messages.
- ✓ Use clear and consistent logging.
- ✓ Set safe file permissions (e.g. `chmod 700 script.sh`).
- ✓ Test scripts in a safe environment before using in production.

7. Common mistakes

- ✗ Not validating input.
- ✗ Not quoting variables.
- ✗ Using unset variables.
- ✗ Ignoring command failures.
- ✗ Hardcoding credentials (e.g. passwords, tokens).
- ✗ Using unnecessary sudo.
- ✗ Deleting files without checks.
- ✗ Writing sensitive information to logs.
- ✗ Assuming `set -euo pipefail` catches everything.

8. Security reminder

- ⚠ Never hardcode credentials in scripts.
- ⚠ Avoid unnecessary sudo.
- ⚠ Validate and sanitize input.
- ⚠ Be careful with file deletion and wildcard patterns.
- ⚠ Do not expose secrets in logs or output files.
- ⚠ Use the minimum permissions required.
- ⚠ Keep sensitive files and scripts protected (e.g. chmod 700).

9. Real-world use

- Build safer and more reliable operational scripts.
- Reduce accidental errors.
- Useful in automation, monitoring, backups, and deployments.
- Helps prevent security incidents.
- Trusted approach used in professional Linux environments.



18 Scheduling Scripts Without Running Them Manually

VERIQTA

Filename: (use any script, e.g. health-report.sh)

1. What this topic covers

- How cron schedules commands automatically.
- Using `crontab -e` and a simple scheduling example.
- Importance of absolute paths and environment differences.
- Sending output to log files.
- How to confirm a scheduled execution.
- Real-world use: schedule a safe health report or a controlled backup task.
- Common failure: a script works interactively but fails when cron runs it.

2. Cron time format and examples

Field	Allowed values	Meaning
Minute	0 - 59	Minute of the hour
Hour	0 - 23	Hour of the day
Day of month	1 - 31	Day of the month
Month	1 - 12	Month of the year
Day of week	0 - 7	Day of the week (0 or 7 = Sunday)

Common examples

Schedule	Cron entry	Meaning
Every day at 2:00am	`0 2 * * *`	Run daily at 2:00am
Every hour	`0 * * * *`	Run at the start of every hour
Every 30 minutes	`*/30 * * * *`	Run every 30 minutes
Every Monday at 1am	`0 1 * * 1`	Run every Monday at 1:00am

3. Example script: health-report.sh (existing script)

```bash
#!/usr/bin/env bash

# Simple health report (short version)
REPORT_DIR="/var/log/server-reports"
mkdir -p "$FEPORT_DIR"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
REPORT_FILE="$REPORT_DIR/health-report-$TIMESTAMP.txt"
{
    echo "Linux Server Health Report"
    echo "Generated: $(date)"
    echo "Hostname: $(hostname)"
    echo ""
    echo "Disk Usage"
    df -h
    echo ""
    echo "Memory Usage"
    free -h
    echo ""
    echo "Top 5 Processes"
    ps aux --sort=-%mem | head -n 5
} > "$REPORT_FILE" 2>&1
echo "Report saved to: $REPORT_FILE"
```

4. Add the script to cron (crontab -e)

1	Open your crontab	`crontab -e`
2	Add the cron entry (use absolute paths)	`0 2 * * * /usr/local/bin/health-report.sh >> /var/log/health-report-cron.log 2>&1`
3	Save and exit	Press ESC, then `:wq`
4	Check your cron jobs	`crontab -l`

5. Example cron entry explained

`0 2 * * * /usr/local/bin/health-report.sh >> /var/log/health-report-cron.log 2>&1`

`0`	Run at minute 0
`2`	Run at 2:00 AM
`* * *`	Every day, every month, every day of week
`/usr/local/bin/health-report.sh`	Absolute path to the script (use full path)
`>> /var/log/health-report-cron.log`	Append output to a log file
`2>&1`	Redirect errors to the same log file

6. Confirm the scheduled execution

- Wait for the scheduled time or run a test entry (e.g. `*/1 * * * *`).
- Check the log file: `cat /var/log/health-report-cron.log`
- Check the report directory: `ls -l /var/log/server-reports/`
- Check system logs (if needed): `grep CRON /var/log/syslog` (Ubuntu)
- Make sure the script ran at the expected time and produced output.

7. Common reasons cron jobs fail

- Using relative paths instead of absolute paths.
- Environment variables (like PATH) are different in cron.
- Script works interactively but fails in cron (missing PATH, permissions).
- Output is not logged, so you cannot see the error.
- Script does not have execute permission.
- Script depends on user-specific files or environment.
- Command runs but produces no output (silent failure).

8. Best practices

- Always use absolute paths for scripts and files.
- Send both output and errors to a log file.
- Keep scripts simple and reliable.
- Ensure the script has execute permission (chmod +x script.sh).
- Test the cron entry and check the logs.
- Use meaningful log file names with timestamps.

9. Real-world use cases

- Schedule a a daily health report.
- Run a controlled backup task.
- Monitor disk usage or log size.
- Run cleanup tasks (e.g. old files).

10. Key takeaways

- Cron runs commands automatically in the background.
- Use absolute paths and log output.
- A script working manually does not guarantee it will work in cron.
- Always confirm the scheduled execution.
- Use cron for safe, repeatable operational tasks.

19 Testing Scripts Before You Trust Them

VERIQTA

Filename: (use any script, e.g. safe-script.sh)

1. What this topic covers

- Use ShellCheck to find common issues in bash scripts.
- Validate syntax before running scripts.
- Test different inputs and failure scenarios.
- Understand and use predictable exit codes.
- Learn how teams review and validate shell scripts before using them in CI/CD pipelines.
- Build confidence that your script works as expected in real environments.

2. Key commands and what they do

Command	What it does
`shellcheck <file>`	Analyzes your script and suggests improvements.
`bash -n <file>`	Checks syntax without running the script.
`bash -x <file>`	Runs with debug output (shows each command).
`echo $?`	Shows the exit code of the last command.
`chmod +x <file>`	Makes the script executable.
`./script.sh <args>`	Runs the script with test inputs.

3. Example script: test-example.sh

```
1  #!/usr/bin/env bash
2
3  # Simple script with input validation and exit codes
4  if [ $# -ne 1 ]; then
5      echo "Usage: $0 <directory>"
6      exit 1
7  fi
8
9  DIR="$1"
10
11 if [ ! -d "$DIR" ]; then
12     echo "Error: Directory '$DIR" does not exist
       (or is not accessible)."
13     exit 1
14 fi
15
16 echo "Success: Directory '$DIR" is accessible."
17 ls -la "$DIR" | head -n 5
18 exit 0
```

4. Test scenarios and example output

Scenario	Command	Expected output	Exit code
Success (valid input)	`./safe-script.sh /var/log`	Script runs successfully	0
Missing input	`./safe-script.sh`	Error: missing input (usage message)	1
Inaccessible file/directory	`./safe-script.sh /root`	Error: permission denied or cannot access	1
Command failure	`./safe-script.sh /notfound`	Error: file not found (or relevant message)	1

5. Using ShellCheck

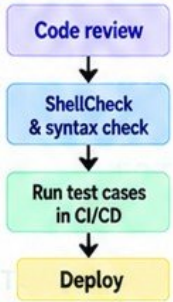
```
$ shellcheck test-example.sh
In test-example.sh line 11:
  if [ ! -d "$DIR" ]; then
       ^-- SC2086: Double quote to prevent globbing
           and word splitting.
$ bash -n test-example.sh
# No output means the syntax is correct.
```

6. Using debug mode to investigate

```
$ bash -x test-example.sh /var/log
+ '[' 1 -ne 1 ']'
+ DIR=/var/log
+ '[' '!' -d /var/log ']'
+ echo 'Success: Directory '/var/log' is accessible.'
+ ls -la /var/log
...
```

7. How teams validate scripts in DevOps (CI/CD)

- ✓ Run ShellCheck as part of automated checks.
- ✓ Validate syntax with bash -n.
- ✓ Use test cases to check expected behavior and exit codes.
- ✓ Run scripts in a clean environment (like a CI runner).
- ✓ Check for security issues (no hardcoded credentials, proper permissions).
- ✓ Review the code (peer review) before merging.
- ✓ Use automated tests to prevent regressions.
- ✓ Only deploy scripts that pass all checks.



8. Common mistakes

- ✗ Skipping ShellCheck.
- ✗ Ignoring warnings.
- ✗ Not validating inputs.
- ✗ Not checking exit codes.
- ✗ Assuming a script works after one successful test.
- ✗ Testing only the success case.
- ✗ Not testing in a clean environment.
- ✗ Using hardcoded paths or credentials.

9. Security reminder

- ⚠ Avoid hardcoded credentials (passwords, API keys).
- ⚠ Do not log sensitive information.
- ⚠ Use proper file permissions (chmod 700 for sensitive scripts).
- ⚠ Validate and sanitize user inputs.
- ⚠ Be careful with commands that delete or modify files.
- ⚠ Do not use unnecessary sudo.
- ⚠ Test in a safe environment before using in production.

10. Key takeaways

- ✓ Use ShellCheck and bash -n before running scripts.
- ✓ Test multiple scenarios, including failures.
- ✓ Use clear and predictable exit codes.
- ✓ Fix issues and warnings.
- ✓ Follow security best practices.
- ✓ Understand how teams validate scripts in CI/CD pipelines.
- ✓ Build trust in your scripts through testing and review.

20 Final Project: Build Your First Bash Automation Toolkit

VERIQTA

Repository name: bash-automation-toolkit

1. What this project covers

- Organize the ten scripts into a Git repository.
- Create a clear README with usage instructions.
- Use consistent filenames and a clean folder structure.
- Demonstrate running the health reporter.
- Investigate a simulated failure and verify the results.
- Document what happened.
- Make it easy for another beginner to clone, run, and verify the scripts safely.

2. Project structure (example)

```
bash-automation-toolkit/
├── README.md                 # Project overview and usage
├── scripts/                  # All 10 scripts
│   ├── 01-system-info.sh
│   ├── 02-disk-usage.sh
│   ├── 03-user-report.sh
│   ├── 04-file-backup.sh
│   ├── 05-process-monitor.sh
│   ├── 06-log-investigator.sh
│   ├── 07-temp-cleaner.sh
│   ├── 08-service-status.sh
│   ├── 09-port-check.sh
│   └── 10-health-report.sh
├── samples/                  # Sample output files
├── logs/                     # Log files (created by scripts)
└── docs/                     # Additional notes (optional)
```

3. README.md (example content)

```
# Bash Automation Toolkit
A collection of 10 practical Bash scripts for Linux
system administration and DevOps practice.

## Requirements
- Linux (Ubuntu recommended)
- Bash 4+
- No external dependencies (except optional tools
  like nc or shellcheck)

## How to use
1. Clone the repository:
   git clone https://github.com/veiqta/bash-automation-toolkit.git
2. Go to the project directory:
   cd bash-automation-toolkit
3. Make scripts executable:
   chmod +x scripts/*.sh
4. Run any script (example):
   ./scripts/10-health-report.sh

## List of scripts
1. 01-system-info.sh       - Show system information
2. 02-disk-usage.sh        - Check disk usage
3. 03-user-report.sh       - User and group report
4. 04-file-backup.sh       - Backup files safely
5. 05-process-monitor.sh   - Monitor running processes
6. 06-log-investigator.sh  - Investigate application logs
7. 07-temp-cleaner.sh      - Find and clean old files (safe)
8. 08-service-status.sh    - Check systemd service status
9. 09-port-check.sh        - Test TCP port connectivity
10. 10-health-report.sh    - Generate server health report
```

4. Run the health reporter (example)

```
$ cd bash-automation-toolkit
$ ./scripts/10-health-report.sh
Generating health report...
Report saved to: /home/user/bash-automation-toolkit/reports/
health-report-2026-10-15_14-30-22.txt
```

5. Sample output (excerpt)

```
================================
Linux Server Health Report
Generated: 2026-10-15 14:30:22
================================
[1] System Information
Hostname        : veriqta-server
Uptime          : 2 days, 3 hours
Kernel          : 6.5.0-101-generic
[2] Disk Usage
Filesystem      Size  Used  Avail Use%  Mounted on
/dev/sda1       100G  45G   50G   48%   /
[3] Memory Usage
Total: 7.7G  Used: 1.2G   Free: 4.1G  Available: 5.8G
[4] Top 5 Processes (by memory)
PID     USER      %MEM     COMMAND
1234    ubuntu     5.0     python3
567     root       3.1     java
890     ubuntu     2.4     nginx
```

6. Investigate a simulated failure

1. Break the script (example: wrong path to a file).
2. Run the script and observe the error message.
3. Check the exit code: `echo $?`
4. Fix the issue and run again.
5. Verify the output and document what happened.

7. Your task

- [x] Create the repository with all 10 scripts.
- [x] Write a clear README with setup and usage.
- [x] Include sample output files in the samples/ folder.
- [x] Run the health reporter and save the report.
- [x] Simulate one failure, investigate and fix it.
- [x] Document the steps and results.
- [x] Test that another beginner can clone and run it successfully.

8. Success checklist

- [x] Repository created with clear structure.
- [x] All scripts are executable.
- [x] README explains how to use each script.
- [x] Health report runs and generates a file.
- [x] Failure scenario is documented.
- [x] Another beginner can follow the README and eproduce the results.

9. Common mistakes

- [x] Missing execute permission (chmod +x).
- [x] Using relative paths that do not work.
- [x] Not providing clear instructions.
- [x] Forgetting to handle errors.
- [x] Not testing on a clean environment.

10. Real-world value

- Reusable automation scripts.
- Helps in server troubleshooting.
- Demonstrates real DevOps skills.
- Can be used in personal projects or team environments.

11. Key takeaways

- [x] Organize scripts in a Git repository.
- [x] Provide clear documentation.
- [x] Test scripts before sharing.
- [x] Handle errors and validate results.
- [x] Make it easy for others to use and learn.
- [x] This is the foundation for larger automation projects in real environments.

07 Script 2: Disk Space Warning System

VERIQTA

Filename: disk-check.sh

1. What this script does

- Checks filesystem usage and prints a warning when usage exceeds a configured threshold.
- Uses `df -P`, numeric comparisons, and `if` conditions.
- Helps you identify a filesystem approaching capacity before an application runs out of disk space.
- Distinguishes filesystem capacity from available space and explains what the threshold measures.

2. Key commands and what they do

Command	What it does
`df -P`	Shows disk usage in a consistent POSIX format.
`df -P /`	Shows usage for a specific filesystem (e.g. /).
`awk`	Extracts specific columns (e.g. usage %).
`tr -d '%'`	Removes the percentage sign.
`if [ ... ]`	Compares numeric values.
`$(( ))`	Performs arithmetic comparison.

3. Example script: disk-check.sh

```
#!/usr/bin/env bash

# Disk space warning system
# Usage: ./disk-check.sh [threshold] [path]
# Example: ./disk-check.sh 80 /

THRESHOLD=${1:-80}       # default 80%
PATH_TO_CHECK=${2:-/}    # default /

# Get usage percentage (remove %)
USAGE=$(df -P "$PATH_TO_CHECK" | awk 'NR==2 {print $5}' | tr -d '%')

echo "Checking filesystem: $PATH_TO_CHECK"
echo "Usage: ${USAGE}% (Threshold: ${THRESHOLD}%)"

if [ "$USAGE" -ge "$THRESHOLD" ]; then
    echo "WARNING: Filesystem $PATH_TO_CHECK is ${USAGE}% used"
    exit 1
else
    echo "OK: Filesystem $PATH_TO_CHECK is ${USAGE}% used"
    exit 0
fi
```

4. How to run the script

1	Make the script executable	`chmod +x disk-check.sh`
2	Run with default settings (80% threshold, /)	`./disk-check.sh`
3	Run with a custom threshold	`./disk-check.sh 90`
4	Check a different path	`./disk-check.sh 80 /var`
5	Check multiple filesystems	`./disk-check.sh 75 /home`

5. Example output

```
$ ./disk-check.sh
Checking filesystem: /
Usage: 58% (Threshold: 80%)
OK: Filesystem / is 58% used
```

Below threshold (exit code 0)

```
$ ./disk-check.sh 70 /var
Checking filesystem: /var
Usage: 82% (Threshold: 70%)
WARNING: Filesystem /var is 82% used
```

Above threshold (exit code 1)

6. Understanding the output

Column	Meaning	Example
Filesystem	The mounted filesystem or device.	/dev/sda1
Size	Total size of the filesystem.	100G
Used	Space used.	58G
Avail	Available space (free).	42G
Use%	Percentage of space used.	58%
Mounted on	Mount point (directory).	/

7. Troubleshooting and common questions

Issue	Cause and solution
Shows wrong usage	Make sure you are checking the correct path (e.g. /, /var). Use `df -P` to see all filesystems.
Script always says 0%	Check if the path exists and is mounted.
Permission denied	You can check most filesystems without sudo. Use `sudo` only when necessary.
Threshold not working	Ensure you are comparing numbers (remove % sign).
Different from `df -h`	This script uses `df -P` for consistent, parseable output. Values may look slightly different from `df -h`.

8. Key points to remember

- Use `df -P` for reliable, parseable output.
- The threshold compares the percentage used (Use%).
- Capacity is the total size, available space is what is free.
- Choose an appropriate threshold for your environment.
- Monitor important filesystems (/, /var, /home, etc.).

9. Real-world use cases

- Monitor server disk usage before it becomes a critical issue.
- Alert your team when a filesystem is approaching capacity.
- Use in automation and monitoring scripts.
- Helps prevent application failures due to lack of disk space.

10. Challenge for you

- Run the script with different thresholds and paths.
- Test with /, /var, and /home.
- Observe the exit codes (0 or 1).
- Modify the script to check multiple filesystems in a loop.
- Add a custom warning message.
- Try scheduling it with cron (later).